Go-onCARE System — Complete Reviewer & Defense Guide

> **Purpose:** This document explains how the entire system works, feature by feature, so you can confidently answer any question during your capstone defense.

TABLE OF CONTENTS

1. System Overview
2. Glossary — Technical Terms Explained
3. Technology Stack
4. Database Design
5. How the App Starts (Login Flow)
6. Role-Based Access Control
7. Feature Breakdown
    - Dashboard
    - Patient Management
    - Appointment Scheduling
    - Clinical & POS (Queue + Billing)
    - Employee Management
    - HR Module
    - Data Analytics
    - Settings & Administration
    - Activity Log
    - Global Search
    - Payroll & Finance Module
8. How the Backend Works
9. How the UI Works
10. Database Relationships Explained
11. Doctor Data Isolation
12. Input Validation System
13. System Analysis — What Makes Sense & What Doesn't
14. Common Defense Questions & Answers
15. Real-Life System Flow Simulation

---

1. SYSTEM OVERVIEW

Go-onCARE is a desktop clinic management system for a **walk-in outpatient clinic** that handles:

- Patient registration and medical records (with address and civil status)
- **Walk-in only** appointment scheduling (always today, auto-confirmed, auto-queued)
- Clinical patient queue with **nurse triage** (vitals + nurse notes) and doctor consultation
- Billing / Point-of-Sale with invoicing and payments
- Employee and HR management (payroll, leave requests)
- **Doctor weekly availability scheduling** (compact day-select-then-time picker)
- Analytics and reporting
- Activity logging (audit trail)

It uses **6 user roles** — Admin, HR, Receptionist, Nurse, Doctor, Finance — each seeing only the pages and buttons relevant to their job.

Walk-In Clinic Flow: Patient arrives → Receptionist creates appointment (always today) → Auto-synced to queue as "Waiting" → Nurse calls next patient, records vitals & triage notes → Doctor calls next patient, conducts consultation → Doctor completes visit → Auto-invoice generated.

---

2. GLOSSARY — TECHNICAL TERMS EXPLAINED

Read this section first. Every technical word used later in the document is explained here in plain language.

Programming Concepts

Term	Plain English Explanation
CRUD	Stands for **Create, Read, Update, Delete** — the 4 basic operations you can do with data. Example: Add a patient (Create), view patient list (Read), change patient phone (Update), remove patient (Delete). Almost every feature in the system is CRUD.
Backend	The "brain" of the app — the code that talks to the database, processes data, and enforces rules. The user never sees the backend. It runs behind the scenes. In this project, the backend is all the Python files inside the `backend/` folder.
Frontend / UI	The part the user actually sees and clicks — buttons, tables, forms, popups. In this project, the UI is all the Python files inside the `ui/` folder.
Class	A blueprint for creating objects. Think of it like a template. Example: `PatientDialog` is a class — every time you click "Add Patient", a new dialog object is created from that blueprint.
Object	An instance of a class. If `PatientDialog` is the blueprint, then the actual popup window you see is the object.
Method / Function	A block of code that does one specific thing. Example: `add_patient()` is a method that inserts a new patient into the database. Methods live inside classes.
Parameter / Argument	Data you pass into a method. Example: `add_patient(data)` — `data` is the parameter (a dictionary containing name, phone, etc.).
Dictionary (dict)	A Python data structure that stores key-value pairs, like a mini-database. Example: `{"name": "Maria Santos", "phone": "+639171234567", "status": "Active"}`. You access values by key: `data["name"]` returns `"Maria Santos"`.
List	An ordered collection of items. Example: `["Active", "Inactive"]` is a list of two strings. You access items by position: `list[0]` returns `"Active"`.
Mixin	A class that adds specific functionality to another class. Instead of putting ALL the code in one giant file, we split it into mixins: `PatientMixin` handles patients, `AppointmentMixin` handles appointments, etc. Then we combine them: `class AuthBackend(PatientMixin, AppointmentMixin, ...)`. Each mixin "mixes in" its methods. Think of it like Lego blocks — each block adds a feature, and you snap them together.
Inheritance	When a class gets all the methods from a parent class. Example: `HREmployeeDialog` inherits from `EmployeeDialog` — it gets all the employee form fields, then adds salary and emergency contact fields on top. The child class reuses the parent's code without rewriting it.
Multiple Inheritance	When a class inherits from MORE than one parent. This is how mixins work: `AuthBackend` inherits from `DatabaseBase`, `PatientMixin`, `AppointmentMixin`, etc. — getting all their methods at once.
Import	Loading code from another file so you can use it. `from backend.patients import PatientMixin` means "bring in the PatientMixin class from the patients.py file so I can use it here."
Module	A single Python file (`.py`). Each file in the project is a module.
Package	A folder of Python files with an `__init__.py`. The `backend/` and `ui/` folders are packages.

Database Concepts

Term	Plain English Explanation
Database	An organized collection of data stored on your computer. Think of it as a big spreadsheet application with many sheets (tables). We use MySQL.
Table	Like a single spreadsheet sheet — has rows (records) and columns (fields). Example: the `patients` table has one row per patient.
Row / Record	One entry in a table. One row in `patients` = one patient (Maria Santos, Female, O+, Active).
Column / Field	A specific piece of data in a table. `first_name`, `phone`, `status` are columns in the `patients` table.

Primary Key (PK)	A unique ID for each row. `patient_id` is the primary key of the `patients` table. No two patients can have the same `patient_id`. The database auto-generates it (AUTO_INCREMENT).
Foreign Key (FK)	A column that points to another table's primary key. It creates a link between tables. Example: `appointments.patient_id` is a foreign key pointing to `patients.patient_id`. This means every appointment MUST belong to a real patient. If you try to create an appointment for patient_id=999 but that patient doesn't exist, the database will reject it.
JOIN	A SQL command that combines data from two or more tables based on a related column. Example: To show "Maria Santos has an appointment with Dr. Ana Reyes", you JOIN the `appointments` table with `patients` and `employees` tables. Without JOINs, you'd only see ID numbers.
LEFT JOIN	A JOIN that keeps ALL rows from the left table, even if there's no match in the right table. Example: `patients LEFT JOIN appointments` shows ALL patients, even those with zero appointments (their appointment columns would be NULL).
SELECT	The SQL command to READ data from the database. `SELECT * FROM patients` means "get all columns from the patients table."
INSERT	The SQL command to CREATE a new record. `INSERT INTO patients (first_name) VALUES ('Maria')` adds a new patient.
UPDATE	The SQL command to MODIFY an existing record. `UPDATE patients SET phone='123' WHERE patient_id=1` changes patient #1's phone.
DELETE	The SQL command to REMOVE a record. `DELETE FROM patients WHERE patient_id=1` removes patient #1.
WHERE	A filter for SQL queries. `SELECT * FROM patients WHERE status='Active'` only returns active patients. Like an IF condition for the database.
GROUP BY	Groups rows that share a value together, used with aggregate functions. `SELECT status, COUNT(*) FROM patients GROUP BY status` returns the count of patients per status (e.g., Active: 10, Inactive: 3).
COUNT / SUM / AVG	Aggregate functions. COUNT counts rows, SUM adds up numbers, AVG calculates the average. Used in dashboard KPIs and analytics.
LIKE	Pattern matching in SQL. `WHERE name LIKE '%Santos%'` finds any name containing "Santos" anywhere. The `%` means "any characters." Used in the search feature.
INDEX	A database optimization that makes searches faster. Like a book's index — instead of reading every page to find "Hypertension," you check the index. We have indexes on `appointment_date`, `status`, `patient_id`, etc.
Transaction	A group of SQL queries that must ALL succeed or ALL fail. Example: creating an invoice needs INSERT into `invoices` AND INSERT into `invoice_items`. If the second INSERT fails, the first one is rolled back (undone). This prevents half-created data.
Schema	The structure/design of the database — which tables exist, what columns they have, how they relate. The `carecrud.sql` file defines the schema.
Seed Data	Pre-loaded sample data inserted when the database is first created. Our seed data includes 5 user accounts, 8 employees, sample patients, services, departments, etc. Used for testing.
CASCADE	Automatic chain reaction when deleting. If `ON DELETE CASCADE` is set on a foreign key, deleting a patient automatically deletes their conditions too. We do most cascading manually in code for more control.
NULL	Means "no value" or "empty." Different from zero or blank string. Example: a patient with no discount has `discount_type_id = NULL`.
ENUM	A column type that only allows specific values. Example: `status ENUM('Active', 'Inactive')` means status can ONLY be "Active" or "Inactive" — nothing else.
AUTO_INCREMENT	The database automatically assigns the next number. Patient #1, #2, #3... You never set the ID manually.

UI / PyQt6 Concepts

Term	Plain English Explanation
PyQt6	A Python library for building desktop application windows with buttons, tables, forms, etc. Qt is the underlying C++ framework; PyQt6 is the Python wrapper. The "6" is the version number.
Widget	Any visible element on screen — a button, a text field, a label, a table, a checkbox. Everything you see is a widget. In PyQt6, all widgets inherit from `QWidget`.

Layout	An invisible container that arranges widgets. `QVBoxLayout` stacks widgets vertically (top to bottom). `QHBoxLayout` arranges them horizontally (left to right). `QFormLayout` creates label-field pairs (like a form).
Signal / Slot	Qt's event system. A **signal** is emitted when something happens (button clicked, text changed). A **slot** is the function that runs in response. `button.clicked.connect(self.save)` means "when button is clicked, run the save() method."
QDialog	A popup window. When you click "Add Patient," a QDialog appears with the form. It blocks interaction with the main window until closed (modal).
QComboBox	A dropdown selector. Used for selecting role, department, status, blood type, etc.
QTableWidget	A table widget with rows and columns. Used to display patient lists, appointment lists, invoices, etc.
QStackedWidget	Shows one page at a time, like a deck of cards. The main content area uses this — clicking "Patients" in the sidebar shows page 1, clicking "Appointments" shows page 2, etc.
QSS (Qt Style Sheets)	CSS-like styling for Qt widgets. Controls colors, borders, fonts, padding, rounded corners. Written in `.qss` files. Example: `QPushButton { background-color: #388087; color: white; border-radius: 10px; }`
Object Name	An ID you give to a widget so QSS can target it specifically. `button.setObjectName("dialogSaveBtn")` lets you style it with `QPushButton#dialogSaveBtn { ... }`. Like CSS class selectors.
Event Filter	A way to intercept events (keyboard, mouse, focus) before they reach a widget. Used for the phone input: when the text field gets focus, the surrounding frame's border turns teal.

General Software Concepts

Term	Plain English Explanation
API	Application Programming Interface — a set of methods/functions you can call. The backend is essentially an API for the UI: the UI calls `backend.get_patients()` and gets data back.
Audit Trail	A record of who did what and when. The `activity_log` table is the audit trail. Important for accountability in medical systems.
Validation	Checking that user input is correct before saving. Examples: phone must be 10 digits, name can't be empty, date can't be in the past. Prevents bad data from entering the database.
Regex (Regular Expression)	A pattern for matching text. `^\d{10}$` means "exactly 10 digits." Used for phone validation. `^` = start, `\d` = digit, `{10}` = exactly 10, `$` = end.
Plaintext Password	Storing passwords as-is without encryption. This is a **security weakness** in this system. In production, passwords should be hashed (scrambled) so even if the database is stolen, passwords can't be read.
KPI	Key Performance Indicator — important numbers that show how things are going. Examples: Today's Revenue, Active Patients, Appointment Count. Displayed on the Dashboard.
POS	Point of Sale — the billing/payment system where transactions happen. In this app, it's the Billing tab of the Clinical page where invoices are created and payments are processed.
Cascading Delete	When deleting one record requires deleting all related records first. Deleting a patient means first deleting their invoice items, invoices, queue entries, appointments, and conditions. Like pulling a thread that unravels connected things.
Refactoring	Restructuring existing code without changing what it does. Making it cleaner, more organized, or more efficient.

---

3. TECHNOLOGY STACK

Layer	Technology	Why
Programming Language	Python	Easy to read, large library ecosystem
Desktop UI Framework	PyQt6	Cross-platform GUI toolkit, professional-looking widgets
Database	MySQL	Relational database, supports foreign keys, transactions, and JOINs
Database Connector	mysql-connector-python	Official MySQL driver for Python

Styling	QSS (Qt Style Sheets)	CSS-like, controls how every widget looks
Charts	Custom QWidget painting	Drawn using QPainter for bar charts and pie charts

Project Structure (v20 — Clean)

```
CareCRUDV1/
├── main.py                  ← Entry point (starts the app)
├── run.bat                  ← Quick launcher (Windows)
├── requirements.txt         ← Python dependencies
├── .gitignore               ← Git ignore rules
│
├── backend/                 ← Business logic & database layer
│   ├── __init__.py          ← Combines all backend mixins into AuthBackend
│   ├── base.py              ← Database connection + helper methods (fetch, exec, exec_many)
│   ├── db_config.py         ← MySQL connection credentials
│   ├── auth.py              ← Login, password management
│   ├── patients.py          ← Patient CRUD operations
│   ├── appointments.py      ← Appointment CRUD + conflict checking
│   ├── clinical.py          ← Queue management, invoicing, services
│   ├── employees.py         ← Employee CRUD + leave + payroll + attendance
│   ├── dashboard.py         ← Dashboard KPI queries + financial summary
│   ├── analytics.py         ← Charts and report data
│   ├── settings.py          ← Admin settings, discount types, cleanup
│   └── search.py            ← Global search across tables
│
├── ui/                      ← Presentation layer (PyQt6)
│   ├── __init__.py
│   ├── auth_window.py       ← Login screen (split-panel with brand panel)
│   ├── main_window.py       ← Main app window (sidebar + stacked pages)
│   ├── styles.py            ← Reusable UI helper functions (make_card, make_banner, etc.)
│   ├── validators.py        ← Input validators (NameValidator, PhoneDigitsValidator, PriceValidator)
│   ├── icons.py             ← SVG icon data for sidebar/buttons
│   ├── styles/              ← Stylesheets & SVG assets
│   │   ├── auth.qss         ← Login screen stylesheet
│   │   ├── main.qss         ← Main app stylesheet
│   │   ├── icon-appointment.svg
│   │   ├── icon-employee.svg
│   │   ├── icon-patient.svg
│   │   ├── icon-service.svg
│   │   ├── calendar.svg
│   │   ├── arrow-up.svg
│   │   └── arrow-down.svg
│   └── shared/              ← All page widgets & dialogs
│       ├── dashboard_page.py       ← Role-specific dashboard (Admin/Doctor/Nurse/HR/Finance)
│       ├── patients_page.py        ← Patient list page
│       ├── patient_dialogs.py      ← Add/Edit Patient (V3), Profile, Merge dialogs
│       ├── appointments_page.py    ← Appointment list + Doctor Availability tab
│       ├── appointment_dialog.py   ← New Walk-in / Edit Appointment (V3, side-by-side schedule)
│       ├── clinical_page.py        ← Queue + Billing + Services tabs
│       ├── clinical_dialogs.py     ← Vitals, Invoice, Payment, Service, BulkPrice dialogs (V3)
│       ├── employees_page.py       ← Admin employee management page
│       ├── employee_dialogs.py     ← Add/Edit Employee (V3) + Profile dialog
│       ├── hr_employees_page.py    ← HR module (Employees, Leave, Payroll, User Accounts)
│       ├── hr_employee_dialogs.py  ← HR-specific Employee, Profile, UserAccount dialogs
│       ├── payroll_page.py         ← Finance payroll approval/rejection page
│       ├── analytics_page.py       ← Data analytics (Admin hospital-wide / Doctor own stats)
│       ├── settings_page.py        ← Settings & Admin tools (password, discounts, cleanup)
│       ├── activity_log_page.py    ← Activity log viewer with filters
│       └── chart_widgets.py        ← Custom BarChart, PieChart, HBarChart widgets
│
├── database/                ← SQL files
│   ├── carecrud.sql         ← Full database schema + seed data
│   ├── sample_data.sql      ← Additional sample data
│   └── drop_all_data.sql    ← Reset script
│
├── SYSTEM_REVIEWER.md       ← This document (defense reviewer guide)
├── SYSTEM_REVIEWER.pdf      ← PDF export of this document
├── USER_MANUAL.md           ← End-user manual
└── USER_MANUAL.pdf          ← PDF export of user manual
```

---

4. DATABASE DESIGN

All Tables (20 total)

Lookup Tables (store fixed reference data)

Table	Purpose	Key Columns
`departments`	Hospital departments	department_id, department_name
`roles`	User roles (Admin, Doctor, Nurse, Receptionist, HR, Finance)	role_id, role_name
`services`	Medical services with prices	service_id, service_name, price, category, is_active
`payment_methods`	How patients pay (Cash, GCash, etc.)	method_id, method_name

`standard_conditions`	Common medical conditions list	condition_id, condition_name
`discount_types`	Discount categories (Senior, PWD)	discount_id, type_name, discount_percent, legal_basis, requires_id_proof

Main Tables (core data)

Table	Purpose	Key Columns
`users`	Login accounts	user_id, email, password, full_name, role_id, must_change_password
`user_preferences`	User settings	pref_id, user_email, dark_mode
`employees`	Staff records	employee_id, first_name, last_name, role_id, department_id, phone, email, hire_date, status, salary
`patients`	Patient records	patient_id, first_name, last_name, sex, date_of_birth, phone, email, address, civil_status, blood_type, discount_type_id, id_proof_path, status
`patient_conditions`	Patient medical conditions	condition_id, patient_id, condition_name

Transaction Tables (things that happen)

Table	Purpose	Key Columns
`appointments`	Scheduled visits	appointment_id, patient_id, doctor_id, service_id, date, time, status
`doctor_schedules`	Doctor weekly availability	schedule_id, doctor_id, day_of_week, start_time, end_time (UNIQUE per doctor+day)
`queue_entries`	Daily patient queue	queue_id, patient_id, doctor_id, appointment_id, blood_pressure, height_cm, weight_kg, temperature, nurse_notes, status, updated_at
`invoices`	Bills	invoice_id, patient_id, total_amount, amount_paid, status, discount_percent
`invoice_items`	Line items on each bill	item_id, invoice_id, service_id, quantity, unit_price, subtotal
`activity_log`	Audit trail	log_id, user_email, action, record_type, record_detail
`leave_requests`	Employee leave applications	request_id, employee_id, leave_from, leave_until, status
`notifications`	Messages to employees	notification_id, employee_id, message, is_read
`paycheck_requests`	HR→Finance paycheck workflow	paycheck_id, employee_id, requested_by, approved_by, amount, period_start, period_end, status

How Tables Connect (Foreign Keys)

```
departments ←── employees ──→ roles
                    ↑
                    │ (doctor_id)
                    ├── doctwor_schedules (weekly availability per doctor)
patients ──→ appointments ──→ services
   ↑              ↑
   │              │
   ├── patient_conditions
   │
   └── invoices ──→ payment_methods
         ↑
         └── invoice_items ──→ services

users ──→ roles
employees ──→ leave_requests
employees ──→ notifications
employees ──→ paycheck_requests (requested_by, approved_by, employee_id)
queue_entries ──→ patients, employees, appointments (+ vitals + nurse_notes)
activity_log (standalone, tracks everything)
```

What is a Foreign Key?

A foreign key is a column that **references** another table's primary key. It ensures data integrity.

- Example: `appointments.patient_id` references `patients.patient_id`
- This means you **can't** create an appointment for a patient that doesn't exist
- If you delete a patient, you must first delete their appointments (cascading delete)

What is a JOIN?

A JOIN combines rows from two tables based on a related column.

```sql
-- Example: Get appointment with patient name and doctor name
SELECT a.appointment_date,
       CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
       CONCAT(e.first_name, ' ', e.last_name) AS doctor_name
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN employees e ON a.doctor_id = e.employee_id
```

This query **joins** the appointments table with patients and employees to get human-readable names instead of just IDs.

---

5. HOW THE APP STARTS (LOGIN FLOW)

Step-by-Step:

1. `main.py` **runs** → Creates the `QApplication` and `AuthWindow` (login screen)
2. **User enters email + password** → Clicks "Sign In"
3. `AuthBackend.login(email, password)` is called:

   ```sql
   SELECT u.*, r.role_name FROM users u
   JOIN roles r ON u.role_id = r.role_id
   WHERE u.email = %s
   ```

   - Checks if email exists
   - Compares password (plaintext match)
   - Returns: success/fail, role name, full name, must_change_password flag
4. **If** `must_change_password = 1` → Forces user to set a new password before continuing
5. **On success** → `login_success` signal emits `(email, role, full_name)`
6. `MainWindow` **is created** with the user's role → sidebar shows only allowed pages
7. **Login window hides**, main window shows

First-Time Password Change:

- When an admin creates a new account, the password is set to something like "doctor123"
- The `must_change_password` flag is set to `1`
- On first login, a dialog forces the user to pick a new password
- After changing, `must_change_password` is set to `0`

---

6. ROLE-BASED ACCESS CONTROL

What Each Role Can See:

Page	Admin	Doctor	Nurse	Receptionist	HR	Finance
Dashboard	✅	✅	✅	✅	✅	✅
Patients	✅ Full CRUD	👁️ View own	👁️ View profiles	✅ Full CRUD	❌	❌
Appointments	✅ Full CRUD	👁️ View/Confirm own	❌	✅ Full CRUD + Doctor Availability	❌	❌
Clinical Queue	✅ View	✅ Call Next/Complete	✅ Start Triage/Record Vitals & Notes	❌	❌	❌
Billing/POS	✅ View	❌	❌	✅ Create/Pay/Void invoices	❌	❌

Services & Pricing	✅ Full CRUD	❌	❌	❌	❌	❌
Employees (Admin)	✅ Full CRUD	❌	❌	❌	❌	❌
HR Module	✅ + User Accounts	❌	❌	❌	✅	❌
Payroll	❌	❌	❌	❌	❌	✅ Approve/Reject
Analytics	✅ Full	✅ Own stats	❌	❌	❌	❌
Activity Log	✅ Full	✅	❌	✅	✅ Full	✅
Settings	✅ Full + DB mgmt	✅ Profile only	✅ Profile only	✅ Profile only	✅ Profile only	✅ Profile only

How It's Implemented:

1. **Sidebar filtering**: `main_window.py` checks the role and only adds allowed menu items
2. **Page-level**: Each page's `__init__` receives the `role` parameter and hides/shows widgets
3. **Button-level**: Individual buttons check `self._role` before appearing

```
# Example: Only Admin and Receptionist see the "Add Patient" button
if self._role not in ("Nurse", "Doctor"):
    banner_btn.setVisible(True)
```

4. **Data-level**: Doctors only see their own patients/appointments (filtered SQL queries)

---

7. FEATURE BREAKDOWN

7.1 Dashboard

What it shows (role-specific):

Each role gets a tailored dashboard with a role-specific banner subtitle explaining their focus:

All roles see:

- Greeting banner with current date/time (updates every second)
- Role-specific subtitle (e.g., Nurse: "Your triage queue — call patients and record their vitals.")
- KPI cards (role-specific — see below)
- Quick action buttons (role-specific — see below)
- My Leave Requests (for all non-HR roles)

Admin/Doctor/Receptionist see:

- KPI cards: Today's Appointments, Active Patients, Today's Revenue, Active Staff (with ↑/↓ delta vs last month)
- Quick actions: New Patient, New Appointment, Clinical Queue, Analytics (filtered by role access)
- Today's Schedule (upcoming appointments table)
- Monthly Visits bar chart (last 6 months)
- Recent Activity log (Admin only)

Nurse sees (triage-focused dashboard):

- KPI cards: **Awaiting Triage** (Waiting count — red if patients need triage, green "all caught up!"), **Triaged Today** (Triaged count — blue "ready for doctor"), **Total In Queue** (Waiting + Triaged + In Progress), **Active Staff**
- Quick actions: **Start Triage** (→ Clinical Queue), **View Patients** (→ Patients page)
- **Patient Queue** card (replaces Today's Schedule) — live mini-table showing active queue entries (Waiting, Triaged, In Progress) with Patient, Time, Doctor, Vitals (✓/—), and Status columns. "Go to Clinical Queue →" button
- **Today's Queue Summary** card (replaces Monthly Visits chart) — visual status breakdown showing Waiting/Triaged/In Progress/Completed counts with colored dots, plus a Completion Rate percentage

HR sees:

- KPI cards: Today's Revenue, Active Staff (appointment and patient KPIs are hidden)
- Quick actions: **Manage Staff** (→ Employees page), **Activity Log** (→ Activity Log page)

How KPIs are calculated:

```sql
-- Today's Appointments
SELECT COUNT(*) FROM appointments WHERE appointment_date = CURDATE()

-- Active Patients
SELECT COUNT(*) FROM patients WHERE status = 'Active'

-- Today's Revenue
SELECT COALESCE(SUM(amount_paid), 0) FROM invoices
WHERE DATE(created_at) = CURDATE() AND status IN ('Paid', 'Partial')

-- Active Staff
SELECT COUNT(*) FROM employees WHERE status = 'Active'
```

Delta % calculation:

```
delta = ((current_month_value - last_month_value) / last_month_value) × 100
```

Auto-refresh: Every 10 seconds, KPIs and schedule are reloaded from the database. The refresh method uses an `isVisible()` guard for efficiency (skips when user is on another page), but accepts a `force=True` parameter that bypasses this check. On initial load and whenever the user navigates to the Dashboard, `refresh(force=True)` is called via `QTimer.singleShot(0, ...)` to ensure the page is fully visible before loading data. This solves the issue where `QStackedWidget` processes show events asynchronously — without the force parameter, the dashboard would appear blank until the user switched away and back.

7.2 Patient Management

CRUD Operations:

Operation	Who Can	What Happens
Create	Admin, Receptionist	Opens form → fills name, sex, DOB, civil status, address, phone, email, emergency contact, blood type, discount, ID Proof image (if required), conditions, status, notes → INSERT into `patients` + `patient_conditions`
Read	All with access	Table shows all patients (Doctor sees only own patients via JOIN with appointments). **Nurse** has View button to see patient profiles (read-only).
Update	Admin, Receptionist	Opens same form pre-filled → UPDATE `patients` + DELETE/re-INSERT `patient_conditions`
Delete	Admin, Receptionist	Confirmation dialog → Cascading delete: invoice_items → invoices → queue_entries → appointments → patient_conditions → patients

Patient Conditions System:

- Standard conditions (Hypertension, Diabetes, etc.) stored in `standard_conditions` table
- When adding a patient, user checks conditions from a checkbox list
- Can also type custom conditions in a text field
- All conditions stored as separate rows in `patient_conditions` table (one row per condition per patient)
- When updating, all old conditions are deleted and new ones inserted

Patient Merge Feature:

- Admin/Receptionist can merge duplicate patients
- Select "Keep" patient and "Remove" patient
- All appointments, invoices, queue entries from "Remove" are transferred to "Keep"
- "Remove" patient's unique conditions are added to "Keep"
- "Remove" patient record is deleted

Discount System:

- Each patient can have one discount type (Senior Citizen 20%, PWD 20%, etc.)
- Stored as `discount_type_id` foreign key

- **ID Proof Requirement**: If the Admin has flagged a discount type as requiring an ID (like a PWD or Senior Citizen card), the UI dynamically shows an image upload field during patient registration. The image is saved locally and a visual thumbnail is shown in the patient profile.
- When an invoice is created, the system looks up the patient's discount and **enforces it from the database** — the UI cannot override this

How the View Profile works:

```
-- 4 separate queries combined into one profile:
-- 1. Patient info (name, sex, dob, phone, etc.)
-- 2. Appointments history (date, doctor, service, status)
-- 3. Invoices history (invoice #, total, paid, status)
-- 4. Queue history (queue #, time, doctor, purpose)
```

7.3 Appointment Scheduling

Walk-In Only Design: This is a **walk-in clinic** — all appointments are for today only. When a receptionist creates an appointment:

1. The date is **always today** (no future scheduling)
2. The status is **always "Confirmed"** (no pending step needed)
3. The appointment is **automatically synced to the queue** as a "Waiting" entry

How creating a walk-in appointment works:

1. User clicks "+ New Appointment"
2. The **V3 UI Dialog** opens with a two-panel layout:
    - **Left Panel (Form)**: Patient (searchable dropdown), Doctor, Time, Service, Notes
    - **Right Panel (Schedule)**: A dynamic table showing the selected doctor's weekly schedule and remaining available slots for the day
3. **Patient search**: Editable combo box with autocomplete — user types a name, suggestions filter as they type
4. **Doctor availability check**: The right panel updates instantly when a doctor is selected, showing their working hours. The system automatically restricts the time picker to fit within their shift and ensures slots haven't passed.
5. **Conflict detection**: Before saving, checks if the doctor already has an appointment at the same time

    ```
    SELECT COUNT(*) FROM appointments
    WHERE doctor_id = %s AND appointment_date = %s AND appointment_time = %s
    AND status != 'Cancelled'
    ```

6. If conflict found, warns user but allows saving anyway (with confirmation)
7. On save: resolves patient name to `patient_id`, INSERT into `appointments` with today's date and "Confirmed" status
8. Queue sync runs automatically — the new appointment appears in the Clinical Queue

Doctor Availability Tab (Receptionist): The Appointments page has a **"Doctor Availability"** tab that shows each doctor's weekly schedule. This helps receptionists assign patients to doctors who are actually working today.

- Shows a table of all doctors with their scheduled days and times
- Data comes from the `doctor_schedules` table
- Managed by Admin in the Employee Management page using a compact schedule picker

Quick Filter Tabs:

- Today, Tomorrow, This Week, This Month, All
- These filter the table rows client-side (no new query needed)

Status Flow (Walk-In):

```
Confirmed → Completed (via queue completion)
      ↘ Cancelled
```

- Walk-in appointments skip the "Pending" step entirely
- **Doctor** can: View details, Cancel (with reason)
- **Admin/Receptionist** can: Create, Edit all fields, Delete

---

7.4 Clinical & POS

This is the most complex page with **3 tabs**:

Tab 1: Patient Queue (Admin, Doctor, Nurse)

How the queue works:

1. When the page loads, it **syncs** today's confirmed appointments to the queue:

```
-- For each confirmed appointment today that's not already in the queue:
INSERT INTO queue_entries (patient_id, doctor_id, appointment_id, queue_time, purpose, status)
```

2. Queue shows **9 columns**: Queue #, Patient, Time, Doctor, Purpose, Vitals, Nurse Notes, Status, Actions
3. **Queue Status Flow (5 states):**

```
Waiting → [Nurse triages] → Triaged → [Doctor calls] → In Progress → Completed/Cancelled
```

   - **Waiting**: Patient is in queue, not yet seen by anyone
   - **Triaged**: Nurse has recorded vitals — patient is ready for the doctor
   - **In Progress**: Doctor is currently consulting the patient
   - **Completed/Cancelled**: Visit finished or cancelled
4. **Nurse workflow (Triage):**
   - Nurse clicks **"Start Triage"** → system finds the first "Waiting" entry (Nurse can only pick "Waiting" patients)
   - The triage vitals dialog opens automatically after calling the patient
   - Nurse records **vitals** (blood pressure, height, weight, temperature) and **triage notes** (free-text observations)
   - **On save, the system automatically sets the patient's status to "Triaged"** — this is the key handoff to the doctor
   - Nurse can also click **"Triage"** on any specific Waiting patient, or **"Update Vitals"** on Triaged/In Progress patients
   - The "Start Triage" button is **disabled** when no Waiting patients exist (tooltip: "No patients waiting for triage")
5. **Doctor workflow:**
   - Doctor clicks **"Call Next"** → system **prefers "Triaged" patients first** (nurse has prepared them), then falls back to "Waiting" if no triaged patients exist. Filtered to the doctor's own patients.
   - This creates a clear **Nurse → Doctor handoff pipeline**: the doctor always gets patients the nurse has already screened
   - Doctor can only have **one** "In Progress" patient at a time
   - After consultation, clicks **"Complete"** → marks queue entry as Completed, marks appointment as Completed, auto-creates an invoice
   - Or clicks **"Cancel"** → marks both queue entry and appointment as Cancelled

Nurse Vitals & Notes Dialog:

```
# When nurse clicks "Triage" (on Waiting patients) or "Update Vitals" (on Triaged/In Progress):
# - Dialog shows: Blood Pressure, Height (cm), Weight (kg), Temperature (°C)
# - Plus a QTextEdit for nurse notes (observations, triage assessment)
# - If updating, all fields are pre-filled with existing values
# - On save: UPDATE queue_entries SET blood_pressure, height_cm, weight_kg, temperature, nurse_notes
# - CRITICAL: If the patient was "Waiting", status is automatically set to "Triaged"
#   This is the Nurse → Doctor handoff — the doctor's "Call Next" prefers Triaged patients
```

Auto-invoice on completion:

```
# When doctor completes a queue entry:
# 1. Look up the appointment's service and price
# 2. Check if patient has a discount (from discount_types table)
# 3. Calculate: subtotal = price × quantity, then apply discount
# 4. INSERT invoice + INSERT invoice_item
# 5. Invoice status = 'Unpaid' (receptionist handles payment later)
```

Wait Time Estimation:

```
-- Average consultation time from last 30 days:
SELECT AVG(TIMESTAMPDIFF(MINUTE, queue_time, completed_time))
FROM queue_entries WHERE status = 'Completed' AND created_at >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)

-- Estimated wait = waiting_count × avg_minutes
```

Tab 2: Billing / POS (Admin, Receptionist)

Invoice table shows: Invoice #, Patient, Services, Total, Paid, Status, Actions

Creating a new invoice manually:

1. Click "+ New Invoice"

2. Select patient (searchable), add service line items (service + quantity)
3. System auto-calculates: subtotal per item, discount (from patient's record), grand total
4. Select payment method, enter amount paid
5. Status auto-determined: amount_paid >= total → "Paid", partial → "Partial", zero → "Unpaid"

How discount is enforced (important!):

```
# The system IGNORES any discount from the UI
# Instead, it looks up the patient's discount from the database:
SELECT discount_percent FROM discount_types
WHERE discount_id = (SELECT discount_type_id FROM patients WHERE patient_id = %s)

# This prevents staff from giving unauthorized discounts
```

Payment processing (Receptionist):

1. Click "Pay" on an unpaid/partial invoice
2. Dialog shows remaining balance
3. Enter payment amount (capped at remaining balance)
4. UPDATE invoice: add to amount_paid, update status

Void (Admin, Receptionist):

- Sets invoice status to "Voided" — the invoice remains in the system for audit trail

Role restrictions:

- **Receptionist**: Can create invoices, pay, and void
- **Admin**: Can view but buttons are hidden (oversight role)
- **Nurse/Doctor**: No access to billing

Tab 3: Services & Pricing (Admin only)

- View all medical services with name, price, category, active/inactive status
- Add new services, edit existing ones
- Bulk price update: change multiple prices at once
- Usage count: shows how many times each service has been invoiced

---

7.5 Employee Management (Admin Side)

Where: Employees page (Admin only)

What it shows:

- 4 stat cards: Total Staff, Doctors, Active, On Leave
- Staff per Department mini-table
- Full employee table with search/filter

CRUD:

Operation	What Happens
Add	Dialog: name, role, department, type, phone (+63 prefix), email, hire date, status, notes → INSERT into `employees` → **Auto-creates a user account**
Edit	Same dialog pre-filled → UPDATE `employees` → syncs email to `users` table
Fire	"Fire" button in edit dialog → cascade deletes all related data (appointments, invoices, queue, notifications, leave requests, doctor_schedules)
View	Profile dialog with tabs: Info, Appointments (last 20), Performance (total appts, completed, revenue)

Doctor Weekly Availability (Compact Schedule Picker): When adding/editing a Doctor, the employee dialog includes a **weekly availability** section:

1. **Day Selection**: 7 toggle buttons (Mon–Sun) — click to enable/disable each day

2. **Time Picker**: For each selected day, set start and end times
3. The UI is compact: select days first, then a shared time picker applies to the current selection
4. On save: DELETE existing schedules → INSERT new ones into `doctor_schedules` table
5. UNIQUE constraint on (doctor_id, day_of_week) prevents duplicate entries

```
-- doctor_schedules table:
-- schedule_id, doctor_id, day_of_week (ENUM Mon-Sun), start_time, end_time
-- Used by: Receptionist's Doctor Availability tab, appointment creation doctor filtering
```

Auto-create user account: When you add a new employee with an email, the system:

1. Splits "Ana Reyes" into first_name="Ana", last_name="Reyes"
2. Creates a user account with password = `{role}123` (e.g., "doctor123")
3. Sets `must_change_password = 1` so they must change it on first login

Duplicate phone check: Before saving, checks if the phone number is already used by another employee.

7.6 HR Module

Where: HR Employees page (HR and Admin)

4 Tabs:

Tab 1: Employees (enhanced with salary/emergency contact)

- Everything the Admin employee page has, PLUS salary and emergency contact fields
- HR stats: Total, Active, On Leave, Inactive, Average Salary, Total Payroll

Tab 2: Leave Management

Leave Request Workflow:

```
Employee submits request (from Dashboard)
          ↓
   Status: "Pending"
          ↓
HR reviews (in Leave Management tab)
          ↓
     ┌────┴────┐
 Approve      Decline
    ↓           ↓
Employee      Employee gets
status →     notification
"On Leave"   with reason
    ↓
All pending/confirmed
appointments during
leave period → Cancelled
    ↓
After leave_until date:
auto_expire_leaves() runs
    ↓
Employee status → "Active"
```

How approval works (SQL):

```sql
-- 1. Update leave request status
UPDATE leave_requests SET status='Approved', hr_decided_by=%s, decided_at=NOW()

-- 2. Update employee status and dates
UPDATE employees SET status='On Leave', leave_from=%s, leave_until=%s

-- 3. Cancel all conflicting appointments
UPDATE appointments SET status='Cancelled', cancellation_reason='Employee on approved leave'
WHERE doctor_id=%s AND appointment_date BETWEEN %s AND %s
AND status IN ('Pending', 'Confirmed')

-- 4. Send notification to employee
INSERT INTO notifications (employee_id, message) VALUES (%s, 'Your leave request has been approved')
```

Auto-expire: Every 5 minutes, `main_window.py` calls `auto_expire_leaves()` which checks:

```sql
UPDATE employees SET status='Active', leave_from=NULL, leave_until=NULL
WHERE status='On Leave' AND leave_until <= CURDATE()
```

Tab 3: Payroll & Staffing

- Department Payroll table: Department, Headcount, Total Salary, Average Salary
- Employment Type Breakdown: Full-time, Part-time, Contract counts
- Read-only summary (no editing)

Tab 4: User Accounts (Admin only)

- View all system login accounts
- Create new account (for existing employee)
- Reset password (sets `must_change_password = 1`)
- Delete account

7.7 Data Analytics

Admin sees (hospital-wide):

Section	What It Shows	SQL Logic
KPI Cards	Monthly Revenue, Total Patients, Appointments, Active Doctors, Avg Visit/Day	Aggregate queries with period comparison
Patient Conditions	Pie chart of most common conditions	`GROUP BY condition_name` from `patient_conditions`
Appointment Status	Pie chart (Pending/Confirmed/Completed/Cancelled)	`GROUP BY status` from `appointments`
Revenue by Department	Pie chart	JOIN invoices → appointments → employees → departments, SUM amount_paid
Patient Demographics	Pie chart by age groups (0-17, 18-35, 36-50, 51-65, 65+)	`CASE WHEN` age calculation from date_of_birth
Patient Retention	New vs Returning patients per month	Subquery: first appointment date per patient
Cancellation Rate	Trend over 6 months	cancelled / total × 100 per month
Doctor Performance	Table: Doctor, Total Appts, Completed, Revenue	GROUP BY doctor
Top Services	Table: Service, Usage Count, Revenue	JOIN invoice_items → services, GROUP BY service
Monthly Revenue Trend	Chart/Table over 6 months	SUM amount_paid per month
Summary with Date Range	Filterable KPI table	WHERE between from_date AND to_date

Doctor sees (own stats only):

- Own KPI cards: Total Appointments, Completed, Cancelled, Revenue, Completion Rate
- Own monthly revenue bar chart
- Own monthly breakdown table

How retention is calculated:

```sql
-- "New" patient = their FIRST appointment ever is in this month
-- "Returning" patient = they had appointments before this month
SELECT DATE_FORMAT(a.appointment_date, '%Y-%m') AS month,
  COUNT(DISTINCT CASE WHEN a.appointment_date = sub.first_date THEN a.patient_id END) AS new_patients,
  COUNT(DISTINCT CASE WHEN a.appointment_date > sub.first_date THEN a.patient_id END) AS returning_patients
FROM appointments a
JOIN (SELECT patient_id, MIN(appointment_date) AS first_date FROM appointments GROUP BY patient_id) sub
```

7.8 Settings & Administration

All roles can:

- Change their own password (current → new → confirm)

Admin can also:

Feature	What It Does
Discount Management	Add/Edit/Delete discount types (Senior 20%, PWD 20%, etc.) with legal basis, and toggle if they require an ID Proof upload
Database Overview	Shows row counts for all 19 tables (auto-refreshes every 10 seconds)
Cleanup: Completed Appointments	Delete completed appointments before a chosen date (cascades to invoices, queue)
Cleanup: Cancelled Appointments	Delete ALL cancelled appointments
Cleanup: Inactive Patients	Delete ALL inactive patients and their linked data
Truncate Table	Empties a table completely (restricted to safe list: queue_entries, invoice_items, invoices, appointments, patient_conditions, activity_log)

Discount deletion safety:

```
# Before deleting a discount type, unlink all patients using it:
UPDATE patients SET discount_type_id = NULL WHERE discount_type_id = %s
# Then delete the discount type:
DELETE FROM discount_types WHERE discount_id = %s
```

7.9 Activity Log

What gets logged: Every important action in the system is recorded with:

- Timestamp, User email, User role, Action, Entity type, Detail text

Actions tracked: Login, Created, Edited, Deleted, Requested, Approved, Declined, Voided, Merged, Synced, Called, Cleaned, Cleared

Example entries:

```
2026-03-09 14:30:22 | admin@carecrud.com | Admin | Created | Patient | Maria Santos
2026-03-09 15:00:00 | ana.reyes@carecrud.com | Doctor | Called | Queue | Called next patient: Juan Dela Cruz
2026-03-09 15:15:00 | nurse@carecrud.com | Nurse | Created | Queue | Recorded vitals for Juan Dela Cruz
2026-03-09 15:30:00 | receptionist@carecrud.com | Receptionist | Created | Invoice | Payment received for Juan Dela Cruz
```

Role filtering:

- **Admin**: Sees everything, can filter by user/action/type/date
- **HR**: Only sees Login activity from Doctors and Receptionists
- **Others**: See the full log but with limited filter options

---

7.10 Global Search

How it works:

1. User types in the search bar at the top of the main window
2. After 2+ characters, backend runs 3 LIKE queries:

```sql
-- Search patients by name, phone, or email
WHERE CONCAT(first_name,' ',last_name) LIKE '%query%' OR phone LIKE '%query%'

-- Search appointments by patient name
WHERE CONCAT(p.first_name,' ',p.last_name) LIKE '%query%'

-- Search employees by name or email
WHERE CONCAT(first_name,' ',last_name) LIKE '%query%' OR email LIKE '%query%'
```

3. Results shown in a popup table (max 10 per category)
4. Employee results hidden from non-admin roles

7.11 Payroll & Finance Module

Purpose: Manages the paycheck workflow between HR and Finance roles.

Workflow:

1. **HR submits a paycheck request** — selects an employee, enters the amount (auto-filled from salary), and specifies the pay period (start/end dates)
2. **Finance reviews the request** — can view all activity done by that employee during the pay period (appointments, logins, etc.) to verify work performed
3. **Finance approves or rejects** — approves valid requests, rejects with a reason if something is wrong
4. **HR disburses the paycheck** — once Finance approves, HR can mark the paycheck as disbursed (paid out)

Files involved:

- `backend/employees.py` — `submit_paycheck_request()`, `get_pending_paycheck_requests()`, `get_all_paycheck_requests()`, `get_employee_activity_for_period()`, `approve_paycheck_request()`, `reject_paycheck_request()`, `disburse_paycheck()`
- `ui/shared/payroll_page.py` — `PayrollPage` (Finance-only page for approval workflow)
- `ui/shared/hr_employees_page.py` — Paycheck request section in the Payroll & Staffing tab (HR submits and disburses)

Database table: `paycheck_requests`

Column	Type	Description
paycheck_id	INT PK	Auto-increment ID
employee_id	INT FK	The employee being paid
requested_by	INT FK	The HR employee who submitted the request
approved_by	INT FK	The Finance employee who approved (NULL until approved)
amount	DECIMAL(12,2)	Paycheck amount
period_start / period_end	DATE	Pay period dates
status	ENUM	Pending → Approved/Rejected → Disbursed
created_at	TIMESTAMP	When the request was submitted

Paycheck request statuses:

- **Pending** — waiting for Finance review
- **Approved** — Finance approved, waiting for HR to disburse
- **Rejected** — Finance rejected (with reason logged)
- **Disbursed** — HR confirmed payout to employee

Access control: Only the Finance role has the Payroll navigation tab. HR manages paycheck requests from the Payroll & Staffing tab within the HR Employees page.

---

8. HOW THE BACKEND WORKS

Architecture: Mixin Pattern

The backend uses **mixin classes** — each file defines a class with specific methods, and they're all combined into one:

```
# backend/__init__.py
class AuthBackend(
    DatabaseBase,      # Connection + helpers (fetch, exec)
    AuthMixin,         # login(), update_password()
    PatientMixin,      # get_patients(), add_patient()
    AppointmentMixin,  # get_appointments(), add_appointment()
    ClinicalMixin,     # get_queue(), add_invoice()
    EmployeeMixin,     # get_employees(), add_employee()
    DashboardMixin,    # get_dashboard_summary()
    AnalyticsMixin,    # get_monthly_revenue()
    SettingsMixin,     # get_discount_types(), truncate_table()
    SearchMixin,       # global_search()
):
    pass
```

Why mixins? Each file handles one feature area, keeping the code organized. But they all share the same database connection from `DatabaseBase`.

Three Core Database Methods:

```
# SELECT queries → Returns list of dictionaries
self.fetch("SELECT * FROM patients WHERE status=%s", ("Active",))
# Returns: [{"patient_id": 1, "first_name": "Maria", ...}, ...]

# Single SELECT → Returns one dictionary
self.fetch("SELECT * FROM patients WHERE patient_id=%s", (1,), one=True)
# Returns: {"patient_id": 1, "first_name": "Maria", ...}

# INSERT/UPDATE/DELETE → Returns row count or ID
self.exec("INSERT INTO patients (first_name) VALUES (%s)", ("Maria",))
# Returns: 1 (the new patient_id)

# Multiple queries in one transaction (all succeed or all fail)
self.exec_many([
    ("INSERT INTO invoices ...", (params,)),
    ("INSERT INTO invoice_items ...", (params,)),
])
```

What is a Transaction?

A transaction groups multiple SQL queries so they **all succeed or all fail**. Example:

- When creating an invoice, you INSERT into `invoices` AND `invoice_items`
- If the invoice_items INSERT fails, the invoice INSERT is **rolled back** (undone)
- This prevents incomplete/broken data

Activity Logging:

```
# Called after every important action:
self.log_activity("Created", "Patient", "Added patient Maria Santos")
# This INSERTs into the activity_log table with the current user's email and role
```

---

9. HOW THE UI WORKS

PyQt6 Concepts You Need to Know:

Concept	What It Is	Example
QWidget	Base class for all UI elements	Every button, label, text field is a QWidget
Layout	Arranges widgets inside a container	QVBoxLayout (vertical), QHBoxLayout (horizontal), QFormLayout (label + field pairs)
Signal/Slot	Event system — when something happens, call a function	`button.clicked.connect(self.on_click)` → when button is clicked, call on_click()
QDialog	Popup window for forms	Add Patient dialog, Appointment dialog
QStackedWidget	Shows one page at a time (like browser tabs)	Main content area switches between Dashboard/Patients/etc.
QTableWidget	Data table with rows and columns	Patient list, appointment list, invoice list
QSS	CSS-like styling for Qt widgets	Colors, borders, fonts, spacing

How a Page Loads:

```
User clicks "Patients" in sidebar
        ↓
main_window._on_nav(index=1)
        ↓
self.stack.setCurrentIndex(1)  ← shows Patients page
        ↓
self._patients_page._load_from_db()  ← refreshes from database
        ↓
Backend.get_patients()  ← SQL query
        ↓
Results populate the QTableWidget rows
```

How a Dialog Works (Add Patient example):

```
User clicks "+ Add Patient"
        ↓
PatientDialog() created  ← form with empty fields
        ↓
User fills in fields, clicks "Save"
        ↓
dialog.accept() called  ← validates phone format
        ↓
dialog.get_data() returns a dictionary:
{
  "name": "Maria Santos",
  "sex": "Female",
  "phone": "+639171234567",
  "email": "maria@email.com",
  "blood_type": "O+",
  "conditions": "Hypertension, Diabetes",
  ...
}
        ↓
Backend.add_patient(data)  ← SQL INSERT
        ↓
Table refreshes to show new patient
```

Styling & V3 UI Structure:

- **V3 UI Architecture**: All major popup dialogs (Add Patient, New Walk-in Appointment, Request Leave, Invoices, Employee Edit) have been upgraded to the **V3 UI Design**. This features a modern gradient header (`qlineargradient` from Teal to Light Blue), embedded SVG icons, polished input fields (`border-radius: 10px`), and a consistent footer button bar. This makes the desktop app feel polished and premium.
- **QSS files** (`auth.qss`, `main.qss`) define the global look — colors, borders, fonts, etc.
- **`styles.py`** has helper functions like `make_card()`, `make_banner()`, `make_table_btn()` that create consistently styled widgets
- **Color palette**: Teal (#388087), Light blue (#BADFE7), Off-white (#F6F6F2), Dark text (#2C3E50)

---

10. DATABASE RELATIONSHIPS EXPLAINED

One-to-Many Relationships:

- One **department** has many **employees**
- One **patient** has many **appointments**
- One **patient** has many **conditions** (in patient_conditions)
- One **invoice** has many **line items** (in invoice_items)
- One **doctor** (employee) has many **appointments**
- One **employee** has many **leave requests**

Why Cascading Deletes?

When you delete a patient, you need to also delete:

1. Their invoice items (because invoice_items → invoices → patients)
2. Their invoices
3. Their queue entries
4. Their appointments
5. Their conditions

The code does this manually in order to respect foreign key constraints:

```python
def delete_patient(self, patient_id):
    queries = [
        ("DELETE FROM invoice_items WHERE invoice_id IN (SELECT invoice_id FROM invoices WHERE patient_id=%s)", (pid,)),
        ("DELETE FROM invoices WHERE patient_id=%s", (pid,)),
        ("DELETE FROM queue_entries WHERE patient_id=%s", (pid,)),
        ("DELETE FROM appointments WHERE patient_id=%s", (pid,)),
        ("DELETE FROM patient_conditions WHERE patient_id=%s", (pid,)),
        ("DELETE FROM patients WHERE patient_id=%s", (pid,)),
    ]
    return self.exec_many(queries)
```

---

11. DOCTOR DATA ISOLATION

The Problem

In a multi-doctor clinic, each doctor should ONLY see data related to their own patients and appointments — not every doctor's data. This is both a **privacy** and **UX** requirement.

How It Works

When a Doctor logs in, the system stores their email (e.g., `ana.reyes@carecrud.com`). This email is used to look up their `employee_id` (which is their `doctor_id` in appointments/queue tables). Every query then filters by this ID.

Isolation Points

Feature	How It's Filtered	Backend Method
Dashboard KPIs	Passes `doctor_email` to count only the doctor's own appointments, patients, and revenue	`get_dashboard_summary(doctor_email=)`
Dashboard Schedule	Only shows upcoming appointments assigned to this doctor	`get_upcoming_appointments(doctor_email=)`
Dashboard Chart	Monthly visits chart shows only this doctor's patient visits	`get_patient_stats_monthly(doctor_email=)`
Patient List	JOINs through appointments to show only patients who have appointments with this doctor	`get_patients_for_doctor(email)`
Appointment List	Filters appointments WHERE doctor's email matches	`get_appointments(doctor_email=)`
Clinical Queue	Filters queue entries WHERE doctor_id matches	`get_queue_entries(doctor_id=)`
Call Next	Only calls the next patient in THIS doctor's queue	`call_next_queue(doctor_id=)`
Analytics	Shows only the doctor's own stats (appointments, revenue, completion rate)	`get_doctor_own_stats(email)`
Global Search	Searches only patients/appointments linked to this doctor	`global_search(doctor_email=)`
Patient Profile	Verifies the patient has appointments with this doctor before returning data	`get_patient_full_profile(patient_id, doctor_email=)`

How the SQL Filtering Works (Example)

```sql
-- Normal query (Admin sees all patients):
SELECT * FROM patients ORDER BY patient_id

-- Doctor-filtered query (Doctor sees only their patients):
SELECT DISTINCT p.* FROM patients p
INNER JOIN appointments a ON p.patient_id = a.patient_id
INNER JOIN employees e ON a.doctor_id = e.employee_id
WHERE e.email = 'ana.reyes@carecrud.com'
```

Defense-in-Depth

The isolation works at **two layers**:

1. **UI layer** — the page checks `if self._role == "Doctor"` and calls the filtered backend method
2. **Backend layer** — the method itself accepts a `doctor_email` or `doctor_id` parameter and alters the SQL query

This means even if someone bypasses the UI check, the backend still enforces the filter.

---

12. INPUT VALIDATION SYSTEM

The Problem

Users may type invalid data into input fields — numbers in name fields, letters in phone fields, malformed emails, excessively long text, etc. This must be caught before it corrupts the database.

Two-Layer Approach

Layer 1: Real-Time Keystroke Blocking (QValidator)

- Prevents invalid characters from being typed at all
- The cursor simply doesn't move when the user presses an invalid key
- Defined in `ui/validators.py`

Validator	What It Allows	Applied To
`NameValidator`	Letters, spaces, hyphens, dots, apostrophes only	Patient name, Employee name
`PhoneDigitsValidator`	Digits only (max 10)	Phone fields
`PriceValidator`	Digits and one decimal point	Service price field

Layer 2: Submit-Time Validation (in `accept()` method)

- Runs when the user clicks "Save"
- Checks for empty required fields, format rules, and minimum lengths
- Shows a warning dialog and blocks save if validation fails

Check	Rule	Where Used
`validate_name()`	Min 2 characters, letters only, required	Patient dialog, Employee dialog
`validate_email()`	Must match `name@domain.ext` pattern	Patient dialog, Employee dialog, Login
`validate_phone_digits()`	Exactly 10 digits	Patient dialog, Employee dialog
`validate_price()`	Must be a valid positive number	Service dialog
`validate_required()`	Cannot be empty/whitespace	Service name, Discount type name

Layer 3: Max Length Enforcement All text fields have `setMaxLength()` to prevent excessively long input:

Field	Max Length
Names	100 characters
Email	150 characters
Emergency contact	150 characters
Service name	150 characters
Cancel/reschedule reasons	300 characters
Other conditions	300 characters
Invoice notes	300 characters
Legal basis	200 characters
Discount type name	100 characters
Phone digits	10 characters
Price	12 characters

Example: What Happens When a User Tries to Type "123" in the Patient Name Field

1. User presses "1" → `NameValidator.validate()` returns `Invalid` → character is rejected
2. The text field doesn't change — the "1" never appears
3. User tries "Maria" → each letter passes validation → text appears normally
4. On Save, `validate_name("Maria")` confirms it's ≥ 2 chars and letters-only → passes

---

13. SYSTEM ANALYSIS

What Makes Sense (and Why)

1. Role System — Well-Designed

Each of the 5 roles (Admin, Doctor, Nurse, Receptionist, HR) has appropriate access. Admin controls everything, Doctor sees only their own data, Nurse handles triage (vitals + notes), Receptionist handles front-desk intake and billing, HR manages personnel. Roles are enforced at three levels: sidebar visibility, page-level button hiding, and SQL query filtering.

2. Walk-In Clinic Workflow — Realistic Outpatient Flow

Patient arrives → Receptionist creates walk-in appointment (always today, auto-confirmed) → Auto-synced to queue as "Waiting" → Nurse triages (records vitals + notes → status auto-changes to **"Triaged"**) → Doctor calls next (prefers Triaged patients → status becomes "In Progress") → Doctor completes → Auto-invoice generated. This mirrors how a real walk-in outpatient clinic operates. The **Triaged status creates a clear Nurse → Doctor handoff** — the doctor always knows which patients the nurse has already screened. The automated steps (auto-confirm, auto-queue sync, auto-triage status, auto-invoice) eliminate manual handoffs.

3. Nurse Triage — Proper Clinical Workflow with Status Handoff

The Nurse role handles patient triage before doctor consultation: recording blood pressure, height, weight, temperature, and triage notes. **When the nurse saves vitals on a "Waiting" patient, the status automatically changes to "Triaged"** — this is the critical handoff. The doctor's "Call Next" then **prefers Triaged patients first**, creating a real Nurse → Doctor pipeline. Nurses can update vitals on already-triaged or in-progress patients without changing status. **The Nurse dashboard is triage-focused**: instead of generic appointment schedules, it shows a live Patient Queue card (Waiting/Triaged/In Progress patients with vitals status), a Today's Queue Summary with status breakdown and completion rate, plus triage-specific KPIs ("Awaiting Triage", "Triaged Today", "Total In Queue").

4. Doctor Weekly Availability — Schedule Management

Doctors have configurable weekly schedules stored in `doctor_schedules`. The compact day-select-then-time picker makes schedule management efficient. Receptionists can view doctor availability when assigning patients, preventing appointments with unavailable doctors.

5. Appointment Conflict Detection — Prevents Double-Booking

Before saving an appointment, the system checks if the doctor already has a non-cancelled appointment at the same date and time. If a conflict exists, it warns the user but allows override (because real clinics sometimes intentionally double-book). Walk-in appointments use today's date automatically.

6. Clinical Queue — Correct State Machine with Triage Step

The queue follows a 5-state flow: **Waiting** → **Triaged** → **In Progress** → **Completed/Cancelled**. The "Triaged" status is the Nurse → Doctor handoff point. A doctor can only have one "In Progress" patient at a time. The Nurse's "Start Triage" picks the earliest Waiting patient; the Doctor's "Call Next" **prefers Triaged patients first** (FIFO among triaged, then falls back to Waiting). This ensures the doctor always sees patients the nurse has already screened. Estimated wait time uses the average consultation duration from the last 30 days.

7. Discount Enforcement — Security-Conscious

When an invoice is created, the system ignores any discount value from the UI and re-fetches the patient's discount from the database. This prevents unauthorized discounts. The discount is based on Philippine laws (RA 9994 for Senior Citizens, RA 7277 for PWDs).

8. Leave Request Auto-Cancellation — Real-World Awareness

When HR approves a leave request, all the doctor's appointments during the leave period are automatically cancelled with the reason "Employee on approved leave." This prevents patients from showing up when their doctor isn't available. The auto-expire mechanism restores the employee to "Active" after their leave ends.

9. Activity Logging — Full Audit Trail

Every important action is logged with timestamp, user, role, action, and details. This is essential for accountability in healthcare and supports Philippine DOH record-keeping requirements.

10. Patient Merge — Practical Deduplication

Receptionists and Admins can merge duplicate patient records. The "Remove" patient's data (appointments, invoices, queue, conditions) is transferred to the "Keep" patient, then the duplicate is deleted. This handles real-world data entry mistakes.

11. Billing System — Multi-Line Items with Partial Payments

Invoices support multiple service line items, quantity, per-item discounts, and partial payments. Status tracks correctly: Unpaid → Partial → Paid. Void preserves the record for audit trail rather than deleting it.

12. Dashboard Real-Time Refresh — Immediate Data Visibility

The dashboard uses a `force` parameter on its refresh method to ensure data loads immediately when navigating to the page, bypassing the `isVisible()` guard that prevents unnecessary background refreshes. Combined with 10-second auto-refresh, administrators always see current data.

What Could Be Improved (and Why)

1. Plaintext Passwords — Security Risk

Passwords are stored as plain strings in the database. If the database were compromised, all passwords would be readable. **Industry standard**: use bcrypt or argon2 to hash passwords.

2. No Time-Slot Duration for Appointments

The conflict check only matches exact time — if two appointments are at 10:00 and 10:05, no conflict is detected. Real clinics need duration-based slots (e.g., 30-minute blocks).

3. No Leave Balance Tracking

Employees can request unlimited leave — there's no concept of leave entitlement (e.g., 15 vacation days, 10 sick days per year). HR has no way to enforce leave quotas.

4. Minimum Password Length Is Only 4 Characters

Healthcare systems handling patient data should enforce stronger password policies (8+ characters with complexity requirements).

---

14. COMMON DEFENSE QUESTIONS & ANSWERS

Architecture & Design

Q: Why did you use a desktop application instead of a web application?

> A: We used PyQt6 for a desktop app because clinics typically run on local machines with local databases. A desktop app provides faster performance, works offline, and doesn't require internet. PyQt6 also gives us native-looking UI elements and full control over the interface.

Q: Why MySQL?

> A: MySQL is a relational database that supports foreign keys, transactions, and complex JOINs — all essential for a system with interconnected data (patients → appointments → invoices). It ensures data integrity through referential constraints.

Q: What design pattern does your backend use?

> A: We use the **Mixin pattern**. Each feature area (patients, appointments, billing, etc.) is a separate Python class. These are combined into one backend class through multiple inheritance. This keeps the code organized — each file handles one responsibility — while sharing the same database connection.

Q: How does your role-based access control work?

> A: Access control happens at three levels: (1) **Sidebar** — only shows pages the role is allowed to see (e.g., Nurse sees Dashboard, Patients, Clinical & POS, Settings), (2) **Page** — buttons and columns are hidden/shown

based on `self._role` (e.g., Nurse gets "Record Vitals" button but not "Complete"), (3) **Data** — SQL queries filter results (e.g., doctors only see their own patients via JOINs with their employee_id).

Database

Q: How many tables does your database have and what are they for?

A: 19 tables total. 6 lookup tables (departments, roles, services, payment_methods, standard_conditions, discount_types), 5 main tables (users, user_preferences, employees, patients, patient_conditions), and 8 transaction tables (appointments, doctor_schedules, queue_entries, invoices, invoice_items, activity_log, leave_requests, notifications).

Q: What are foreign keys and why do you use them?

A: Foreign keys are columns that reference another table's primary key. For example, `appointments.patient_id` references `patients.patient_id`. This ensures you can't create an appointment for a patient that doesn't exist. It maintains data integrity.

Q: How do you handle deleting a patient that has appointments and invoices?

A: We use cascading deletes — we delete related records in order: invoice_items → invoices → queue_entries → appointments → patient_conditions → patients. All within one transaction so if any step fails, everything rolls back.

Q: What indexes do you have and why?

A: We have indexes on frequently filtered columns: appointment_date, doctor_id, patient_id, status fields. Indexes speed up SELECT queries by creating a lookup structure, like a book's index. Without them, MySQL would scan every row.

Features

Q: How does the appointment conflict detection work?

A: Before saving an appointment, we query: "Does this doctor already have a non-cancelled appointment at this date and time?" If yes, we warn the user. They can still force-save (for legitimate double-bookings), but they must acknowledge the conflict.

Q: How does the patient queue work?

A: This is a walk-in clinic, so all appointments are for today. Confirmed appointments are automatically synced to the queue as "Waiting". The queue has 5 statuses: **Waiting** → **Triaged** → **In Progress** → **Completed/Cancelled**. The **Nurse** clicks "Start Triage" to call the next waiting patient — when vitals are saved, the status automatically changes to **"Triaged"**. The **Doctor** then clicks "Call Next" → the system **prefers Triaged patients first** (ones the nurse already screened), setting them to "In Progress". After the consultation, the doctor marks it "Complete" → the system auto-creates an unpaid invoice for the receptionist. Queue is single-day only — it resets daily.

Q: How are discounts calculated?

A: Each patient can have a discount type (e.g., Senior Citizen 20%). When an invoice is created, the system reads the discount **from the database** — not from the UI. This prevents unauthorized discounts. The discount is applied: `total = subtotal × (1 - discount_percent/100)`.

Q: How does the leave request system work?

A: An employee requests leave from the Dashboard. HR sees it in the Leave Management tab and can Approve or Decline. If approved: the employee's status changes to "On Leave", all their appointments during the leave period are auto-cancelled, and a notification is sent. After the leave end date, an auto-expire check restores the employee to "Active" status.

Q: What is the activity log?

A: Every important action (login, create, edit, delete, void, merge, etc.) is recorded with a timestamp, user, role, and description. This creates an audit trail — you can see who did what and when. HR can only see login

activity, while Admin sees everything.

Technical

Q: How does the search functionality work?

A: The global search uses SQL LIKE queries with wildcards: `WHERE name LIKE '%query%'`. This searches for the text anywhere in the name, phone, or email. It searches patients, appointments, and employees simultaneously, limited to 10 results each to keep it fast.

Q: How do you prevent data inconsistency?

A: Three mechanisms: (1) **Foreign keys** prevent orphaned records, (2) **Transactions** (exec_many) ensure multiple related queries all succeed or all fail, (3) **Cascading deletes** remove all related data when a parent record is deleted.

Q: How do you handle the phone number format?

A: Philippine format (+63 followed by 10 digits). The UI shows a styled frame with "+63" as a fixed prefix label, and the user only types the 10 digits. On save, "+63" is prepended. On edit, "+63" is stripped for display. Validation uses regex: `^\d{10}$`.

Q: What happens when the system first connects to the database?

A: The `DatabaseBase.__init__()` runs `_ensure_schema()` which checks if required tables and columns exist. If any are missing (e.g., the `discount_types` table, the `nurse_notes` column in `queue_entries`, or the `updated_at` column), it creates them. This is a basic auto-migration so the app works even on older database versions.

If They Ask "Did You Use AI?"

Be honest. Here are helpful ways to frame it:

"I used AI as a development tool to accelerate the coding process, similar to how developers use Stack Overflow, documentation, and code assistants in industry. I understand how each component works — let me walk you through [specific feature]."

Then demonstrate understanding by explaining the SQL, the data flow, or the role-based logic in detail. **The goal of this document is to give you that understanding.**

QUICK REFERENCE: SQL QUERIES YOU SHOULD KNOW

```sql
-- Get all active patients with their conditions
SELECT p.*, GROUP_CONCAT(pc.condition_name) AS conditions
FROM patients p
LEFT JOIN patient_conditions pc ON p.patient_id = pc.patient_id
WHERE p.status = 'Active'
GROUP BY p.patient_id

-- Get today's appointments with patient and doctor names
SELECT a.*, CONCAT(p.first_name,' ',p.last_name) AS patient_name,
       CONCAT(e.first_name,' ',e.last_name) AS doctor_name,
       s.service_name
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN employees e ON a.doctor_id = e.employee_id
JOIN services s ON a.service_id = s.service_id
WHERE a.appointment_date = CURDATE()

-- Get invoice with line items
SELECT i.*, CONCAT(p.first_name,' ',p.last_name) AS patient_name,
       GROUP_CONCAT(s.service_name) AS services
FROM invoices i
JOIN patients p ON i.patient_id = p.patient_id
JOIN invoice_items ii ON i.invoice_id = ii.invoice_id
JOIN services s ON ii.service_id = s.service_id
GROUP BY i.invoice_id

-- Check appointment conflict
SELECT COUNT(*) FROM appointments
WHERE doctor_id = 1 AND appointment_date = '2026-03-09'
AND appointment_time = '09:00:00' AND status != 'Cancelled'

-- Monthly revenue for last 6 months
SELECT DATE_FORMAT(created_at, '%Y-%m') AS month,
       SUM(amount_paid) AS total_revenue
FROM invoices
```

```sql
WHERE status IN ('Paid', 'Partial')
AND created_at >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
GROUP BY month ORDER BY month

-- Dashboard: today's revenue
SELECT COALESCE(SUM(amount_paid), 0)
FROM invoices
WHERE DATE(created_at) = CURDATE() AND status IN ('Paid', 'Partial')
```

---

15. REAL-LIFE SYSTEM FLOW SIMULATION

> This section walks through a complete day at **Go-onCARE Clinic**, showing exactly what each role does in the system and what happens in the database at every step. Use this to demonstrate the system during your defense.

SCENARIO: A Typical Morning at the Clinic

Date: March 10, 2026 (Tuesday)
Clinic opens at 8:00 AM
Staff on duty: Carlo Santos (Admin), James Cruz (Receptionist), Sofia Reyes (Nurse), Dr. Ana Reyes (Doctor)

8:00 AM — Staff Arrive and Log In

Each staff member opens the app and logs in.

Who	Email	What They See
Carlo (Admin)	admin@carecrud.com	Full dashboard — all KPI cards, today's schedule, recent activity, quick action buttons
James (Receptionist)	james.cruz@carecrud.com	Dashboard — KPI cards, quick action buttons (New Patient, New Appointment), today's schedule
Sofia (Nurse)	sofia.reyes@carecrud.com	**Triage-focused dashboard** — Awaiting Triage / Triaged / In Queue KPIs, Patient Queue card (empty), Queue Summary card, Start Triage + View Patients buttons
Dr. Ana (Doctor)	ana.reyes@carecrud.com	Dashboard — her own KPI cards, her own schedule (empty so far)

What happens in the database:

```sql
-- Each login is recorded in the activity log:
INSERT INTO activity_log (user_email, user_role, action, record_type, record_detail)
VALUES ('admin@carecrud.com', 'Admin', 'Login', 'User', 'User logged in');

INSERT INTO activity_log (user_email, user_role, action, record_type, record_detail)
VALUES ('james.cruz@carecrud.com', 'Receptionist', 'Login', 'User', 'User logged in');

-- ... same for Nurse and Doctor
```

Dashboard auto-refreshes every 10 seconds. The Admin's dashboard shows real-time KPI updates using `refresh(force=True)` on navigation.

8:15 AM — First Patient Arrives (New Patient)

Maria Santos, 45 years old, walks in with a cough. She has never been to this clinic before.

Step 1: Receptionist Registers the Patient

James (Receptionist) clicks **"+ Add Patient"** on the Patients page.

He fills in the form:

Field	Value
First Name	Maria

Last Name	Santos
Sex	Female
Date of Birth	1981-05-12
Civil Status	Married
Address	123 Rizal St, Quezon City
Phone	+63 917 123 4567
Email	maria.santos@email.com
Blood Type	O+
Conditions	☑ Hypertension
Discount	Senior Citizen (20%) — *No, she's 45, so no discount*
Status	Active

What happens in the database:

```sql
-- 1. Patient record is created
INSERT INTO patients (first_name, last_name, sex, date_of_birth, civil_status, address,
                      phone, email, blood_type, status)
VALUES ('Maria', 'Santos', 'Female', '1981-05-12', 'Married', '123 Rizal St, Quezon City',
        '+639171234567', 'maria.santos@email.com', 'O+', 'Active');
-- Returns patient_id = 21 (auto-increment)

-- 2. Medical condition is linked
INSERT INTO patient_conditions (patient_id, condition_name) VALUES (21, 'Hypertension');

-- 3. Activity is logged
INSERT INTO activity_log (user_email, user_role, action, record_type, record_detail)
VALUES ('james.cruz@carecrud.com', 'Receptionist', 'Created', 'Patient', 'Maria Santos');
```

Validation that happened silently:

- `NameValidator` blocked any digits in the name field (real-time keystroke blocking)
- Phone field only accepted 10 digits after the +63 prefix
- Email matched the `name@domain.ext` pattern
- All required fields were filled before the Save button worked

---

Step 2: Receptionist Creates a Walk-In Appointment

James clicks **"+ New Appointment"** on the Appointments page.

He fills in the dialog:

Field	Value
Patient	Maria Santos *(typed "Mar", autocomplete showed her)*
Doctor	Dr. Ana Reyes *(only doctors available today — Tuesday — are shown)*
Time	08:30 AM
Service	General Consultation (₱500)
Notes	Walk-in, cough for 3 days

What the system checks before saving:

1. Is Dr. Ana available on Tuesdays? → Checks `doctor_schedules`:

   ```sql
   SELECT * FROM doctor_schedules WHERE doctor_id = 1 AND day_of_week = 'Tuesday'
   -- Result: start_time=08:00, end_time=17:00 → Yes, available
   ```

2. Does Dr. Ana already have an appointment at 08:30? → Conflict check:

```
SELECT COUNT(*) FROM appointments
WHERE doctor_id = 1 AND appointment_date = '2026-03-10'
AND appointment_time = '08:30:00' AND status != 'Cancelled'
-- Result: 0 → No conflict
```

What happens in the database:

```
-- 1. Appointment is created (always today, always Confirmed for walk-ins)
INSERT INTO appointments (patient_id, doctor_id, service_id, appointment_date,
                          appointment_time, status, notes)
VALUES (21, 1, 1, '2026-03-10', '08:30:00', 'Confirmed', 'Walk-in, cough for 3 days');
-- Returns appointment_id = 101

-- 2. Activity logged
INSERT INTO activity_log (...) VALUES (..., 'Created', 'Appointment', 'Maria Santos with Dr. Ana Reyes');
```

Immediately after saving, the system runs the queue sync:

```
-- Auto-sync: Confirmed appointment today not yet in queue → create queue entry
INSERT INTO queue_entries (patient_id, doctor_id, appointment_id, queue_time, purpose, status)
VALUES (21, 1, 101, '08:30:00', 'General Consultation', 'Waiting');
-- Returns queue_id = 50
```

Maria is now in the queue as "Waiting". The Clinical & POS page updates automatically within 10 seconds.

8:20 AM — Second Patient Arrives (Returning Patient)

Juan Dela Cruz, 67 years old (Senior Citizen, 20% discount), comes in for a follow-up checkup.

James recognizes Juan is already in the system. He goes straight to **"+ New Appointment"**:

- Types "Juan" → autocomplete finds "Juan Dela Cruz"
- Selects Dr. Ana Reyes, 08:45 AM, General Consultation
- Saves → auto-confirmed, auto-queued

Queue now shows:

Queue #	Patient	Time	Doctor	Purpose	Vitals	Nurse Notes	Status
50	Maria Santos	08:30	Dr. Ana Reyes	General Consultation	—	—	Waiting
51	Juan Dela Cruz	08:45	Dr. Ana Reyes	General Consultation	—	—	Waiting

---

8:25 AM — Nurse Begins Triage

Sofia (Nurse) goes to the Clinical & POS page → Queue tab.

She sees both patients waiting. She clicks **"Start Triage"** (the Nurse version of "Call Next").

What happens:

```
-- System finds the first "Waiting" entry (FIFO — oldest queue_time first)
-- Nurse can ONLY pick "Waiting" patients (not Triaged or In Progress)
SELECT queue_id FROM queue_entries
WHERE status = 'Waiting' ORDER BY queue_time ASC LIMIT 1;
-- Result: queue_id = 50 (Maria Santos)
```

The triage dialog opens automatically — the Nurse doesn't need to separately click "Record Vitals."

Activity logged:

```
INSERT INTO activity_log (...) VALUES (..., 'Called', 'Queue', 'Called next patient: Maria Santos');
```

Queue updates to:

Queue #	Patient	Time	Doctor	Purpose	Vitals	Nurse Notes	Status
50	Maria Santos	08:30	Dr. Ana Reyes	General Consultation	—	—	Waiting

51	Juan Dela Cruz	08:45	Dr. Ana Reyes	General Consultation	—	—	Waiting

---

Nurse Records Vitals for Maria

The triage dialog is already open (it opened automatically after "Start Triage"):

Field	Value Sofia Enters
Blood Pressure	140/90
Height (cm)	162.5
Weight (kg)	65.0
Temperature (°C)	37.8
Nurse Notes	"Patient reports persistent dry cough for 3 days. Mild fever. BP slightly elevated — consistent with existing hypertension. Recommend doctor check for respiratory infection."

What happens in the database:

```
-- Vitals are saved AND status automatically changes to "Triaged"
UPDATE queue_entries
SET blood_pressure = '140/90', height_cm = 162.5, weight_kg = 65.0,
    temperature = 37.8, nurse_notes = 'Patient reports persistent dry cough for 3 days...',
    status = 'Triaged', updated_at = NOW()
WHERE queue_id = 50;
```

This is the critical handoff: Maria's status is now **"Triaged"** — the doctor knows the nurse has already screened her.

Queue now shows:

Queue #	Patient	Vitals	Nurse Notes	Status
50	Maria Santos	BP:140/90 H:162.5 W:65.0 T:37.8	Patient reports persistent dry...	**Triaged**
51	Juan Dela Cruz	—	—	Waiting

Sofia can also click "View" on the Patients page to see Maria's full profile (read-only) — checking her medical history and conditions before the doctor sees her.

Nurse Triages the Second Patient

Sofia clicks **"Start Triage"** again → Juan Dela Cruz is called (he's the next "Waiting" patient).

She records his vitals:

Field	Value
Blood Pressure	130/85
Height (cm)	170.0
Weight (kg)	72.5
Temperature (°C)	36.5
Nurse Notes	"Follow-up visit. Vitals stable. No complaints other than routine checkup. Senior citizen — verify discount on file."

On save, Juan's status automatically changes to "Triaged" too.

Queue now shows:

Queue #	Patient	Status
50	Maria Santos	**Triaged**
51	Juan Dela Cruz	**Triaged**

Both patients have been triaged by the nurse and are **ready for the doctor**. The "Start Triage" button is now **disabled** (tooltip: "No patients waiting for triage") since there are no more "Waiting" patients.

8:40 AM — Doctor Begins Consultations

Dr. Ana (Doctor) navigates to Clinical & POS → Queue tab.

She sees both patients with vitals already recorded by the nurse — both marked as **"Triaged"**. She clicks **"Call Next"**.

What happens:

```sql
-- Doctor's "Call Next" PREFERS "Triaged" patients first (nurse has already screened them)
-- Falls back to "Waiting" only if no Triaged patients exist
SELECT queue_id FROM queue_entries
WHERE doctor_id = 1 AND status IN ('Triaged', 'Waiting')
ORDER BY FIELD(status, 'Triaged', 'Waiting'), queue_time ASC LIMIT 1;
-- Result: queue_id = 50 (Maria Santos — Triaged, earliest queue_time)

-- Sets status to "In Progress" (only the Doctor sets this status)
UPDATE queue_entries SET status = 'In Progress', updated_at = NOW() WHERE queue_id = 50;
```

> **Key difference from Nurse:** The Nurse's "Start Triage" only picks "Waiting" patients and auto-opens the vitals dialog. The Doctor's "Call Next" prefers "Triaged" patients (creating a clear Nurse → Doctor pipeline) and sets status to "In Progress".

Dr. Ana reviews Maria's vitals and nurse notes directly in the queue table:

- **BP: 140/90** (elevated — matches her hypertension)
- **Temp: 37.8°C** (mild fever)
- **Nurse Notes:** "Persistent dry cough for 3 days. Mild fever..."

She conducts the consultation, then clicks **"Complete"** on Maria's row.

What Happens When Doctor Clicks "Complete"

This triggers a **chain of 4 database operations in one transaction:**

```sql
-- 1. Mark queue entry as Completed
UPDATE queue_entries SET status = 'Completed', updated_at = NOW() WHERE queue_id = 50;

-- 2. Mark the linked appointment as Completed
UPDATE appointments SET status = 'Completed' WHERE appointment_id = 101;

-- 3. Look up service price and patient discount
SELECT s.price FROM services s
JOIN appointments a ON a.service_id = s.service_id
WHERE a.appointment_id = 101;
-- Result: ₱500.00

SELECT dt.discount_percent FROM discount_types dt
JOIN patients p ON p.discount_type_id = dt.discount_id
WHERE p.patient_id = 21;
-- Result: NULL (Maria has no discount)

-- 4. Auto-create invoice
INSERT INTO invoices (patient_id, total_amount, discount_percent, amount_paid, status, payment_method_id)
VALUES (21, 500.00, 0, 0, 'Unpaid', NULL);
-- Returns invoice_id = 200

INSERT INTO invoice_items (invoice_id, service_id, quantity, unit_price, subtotal)
VALUES (200, 1, 1, 500.00, 500.00);
```

Maria's visit is now complete. An unpaid invoice for ₱500.00 appears in the Billing tab.

Doctor Completes Juan's Visit (WITH Discount)

Dr. Ana completes Juan Dela Cruz's consultation. The same chain runs, but **this time there's a discount:**

```sql
-- Juan is a Senior Citizen → discount_type_id links to "Senior Citizen 20%"
SELECT dt.discount_percent FROM discount_types dt
JOIN patients p ON p.discount_type_id = dt.discount_id
WHERE p.patient_id = 5;
-- Result: 20.00

-- Invoice is created with discount applied FROM THE DATABASE (not from UI)
-- Subtotal: ₱500 × (1 - 20/100) = ₱400.00
INSERT INTO invoices (patient_id, total_amount, discount_percent, amount_paid, status)
VALUES (5, 400.00, 20.00, 0, 'Unpaid');
```

Key security point: The 20% discount was pulled from the database, NOT entered by any user. Even if someone tried to change the discount in the UI, the system ignores it and enforces the database value.

9:00 AM — Receptionist Handles Billing

James (Receptionist) goes to Clinical & POS → Billing tab.

He sees two unpaid invoices:

Invoice #	Patient	Services	Total	Paid	Status
200	Maria Santos	General Consultation	₱500.00	₱0.00	Unpaid
201	Juan Dela Cruz	General Consultation	₱400.00	₱0.00	Unpaid

Processing Maria's Payment

James clicks **"Pay"** on Invoice #200. A dialog opens:

Field	Value
Total	₱500.00
Amount Paid So Far	₱0.00
Remaining Balance	₱500.00
Payment Amount	₱500.00
Payment Method	Cash

What happens:

```sql
UPDATE invoices
SET amount_paid = 500.00, status = 'Paid', payment_method_id = 1
WHERE invoice_id = 200;

INSERT INTO activity_log (...) VALUES (..., 'Created', 'Invoice', 'Payment received for Maria Santos');
```

Juan Pays Partially (Real-World Scenario)

Juan only has ₱200 right now. James enters ₱200:

```sql
UPDATE invoices SET amount_paid = 200.00, status = 'Partial' WHERE invoice_id = 201;
```

Invoice #201 now shows: Total: ₱400 | Paid: ₱200 | Status: **Partial**

Juan can come back later to pay the remaining ₱200. The system tracks partial payments.

9:30 AM — Admin Checks the Dashboard

Carlo (Admin) clicks on Dashboard. The page calls `refresh(force=True)` and immediately shows updated numbers:

KPI Card	Value	Change

Today's Appointments	2	↑ from 0 this morning
Active Patients	22	↑ 1 (Maria is new)
Today's Revenue	₱700.00	₱500 (Maria) + ₱200 (Juan partial)
Active Staff	8	No change

Today's Schedule shows:

Time	Patient	Doctor	Status
08:30	Maria Santos	Dr. Ana Reyes	✅ Completed
08:45	Juan Dela Cruz	Dr. Ana Reyes	✅ Completed

Recent Activity shows:

```
09:01 | james.cruz@carecrud.com | Receptionist | Created | Invoice | Payment received for Juan Dela Cruz
09:00 | james.cruz@carecrud.com | Receptionist | Created | Invoice | Payment received for Maria Santos
08:55 | ana.reyes@carecrud.com  | Doctor       | Created | Queue   | Completed queue entry for Juan Dela Cruz
08:40 | ana.reyes@carecrud.com  | Doctor       | Created | Queue   | Completed queue entry for Maria Santos
08:25 | sofia.reyes@carecrud.com| Nurse        | Created | Queue   | Recorded vitals for Maria Santos
08:15 | james.cruz@carecrud.com | Receptionist | Created | Patient | Maria Santos
```

The dashboard auto-refreshes every 10 seconds, so these numbers stay current throughout the day.

10:00 AM — HR Handles a Leave Request

Meanwhile, Dr. Mark Tan submitted a leave request yesterday from his Dashboard.

Elena (HR) logs in → goes to **HR Module** → **Leave Management tab**.

She sees:

Employee	Type	From	To	Reason	Status
Dr. Mark Tan	Vacation	Mar 15	Mar 20	Family event	Pending

Elena clicks **"Approve"**. This triggers:

```sql
-- 1. Approve the leave request
UPDATE leave_requests SET status = 'Approved', hr_decided_by = 8, decided_at = NOW()
WHERE request_id = 12;

-- 2. Update employee status
UPDATE employees SET status = 'On Leave', leave_from = '2026-03-15', leave_until = '2026-03-20'
WHERE employee_id = 2;

-- 3. Auto-cancel all of Dr. Mark's appointments during leave period
UPDATE appointments SET status = 'Cancelled', cancellation_reason = 'Employee on approved leave'
WHERE doctor_id = 2 AND appointment_date BETWEEN '2026-03-15' AND '2026-03-20'
AND status IN ('Pending', 'Confirmed');

-- 4. Notify Dr. Mark
INSERT INTO notifications (employee_id, message)
VALUES (2, 'Your leave request has been approved');
```

On March 21, 2026, the auto-expire check runs (every 5 minutes):

```sql
-- Leave expired → restore to Active
UPDATE employees SET status = 'Active', leave_from = NULL, leave_until = NULL
WHERE employee_id = 2 AND status = 'On Leave' AND leave_until <= CURDATE();
```

Dr. Mark is automatically back to Active status. No manual action needed.

2:00 PM — Admin Does Housekeeping

Carlo (Admin) goes to **Settings** → **Admin Tools**.

He checks the Database Overview:

Table	Row Count
patients	22
appointments	102
queue_entries	51
invoices	201
activity_log	340
...	...

He runs a cleanup: Delete completed appointments older than January 1, 2026:

```
-- Cascading delete: invoice_items → invoices → queue_entries → appointments
DELETE FROM invoice_items WHERE invoice_id IN
  (SELECT invoice_id FROM invoices WHERE patient_id IN
    (SELECT patient_id FROM appointments WHERE status='Completed' AND appointment_date < '2026-01-01'));
-- ... cascading continues
```

5:00 PM — End of Day Summary

Carlo checks Analytics to see the day's performance:

Metric	Value
Total Appointments Today	2
Completed	2 (100% completion rate)
Revenue Collected	₱700.00
Outstanding Balance	₱200.00 (Juan's remaining)
New Patients	1 (Maria Santos)
Patients Seen by Dr. Ana	2

Dr. Ana checks her own Analytics:

- My Appointments: 2
- Completed: 2
- My Revenue: ₱700.00
- Completion Rate: 100%

COMPLETE SYSTEM FLOW DIAGRAM

```
╔═══════════════════════════════════════════════════════════════╗
║                    GO-ONCARE WALK-IN CLINIC FLOW              ║
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║  PATIENT ARRIVES                                             ║
║       │                                                      ║
║       ▼                                                      ║
║  ┌──────────────────────────────────┐                        ║
║  │ RECEPTIONIST                     │                        ║
║  │ 1. Register patient (if new)     │                        ║
║  │ 2. Create walk-in appointment    │──→ DB: patients,       ║
║  │    (always today, auto-confirmed)│       appointments     ║
║  │ 3. Auto-synced to queue          │──→ DB: queue_entries   ║
║  └──────────────────────────────────┘       (status: Waiting) ║
║                 │                                            ║
║                 ▼                                            ║
║  ┌──────────────────────────────────┐                        ║
║  │ NURSE (Triage)                   │                        ║
║  │ 1. "Start Triage" → next Waiting │                        ║
║  │ 2. Record vitals (BP, H, W, Temp)│──→ DB: queue_entries   ║
║  │ 3. Write triage notes            │       (vitals + nurse_notes)║
║  │ 4. On save → status = "Triaged"  │──→ status: Triaged     ║
║  │    (auto handoff to doctor)      │    (ready for doctor)  ║
║  └──────────────────────────────────┘                        ║
║                 │                                            ║
║                 ▼                                            ║
║  ┌──────────────────────────────────┐                        ║
```

```
│  │  DOCTOR (Consultation)              │                          │
│  │  1. "Call Next" → prefers Triaged   │──→ status: In Progress   │
│  │     (nurse-screened patients first) │                          │
│  │  2. Reviews vitals & nurse notes    │                          │
│  │  3. Conducts examination            │                          │
│  │  4. Clicks "Complete"               │──→ DB: queue_entries     │
│  │     • Queue entry → Completed       │       (status: Completed) │
│  │     • Appointment → Completed       │──→ DB: appointments      │
│  │     • Auto-creates invoice          │──→ DB: invoices +        │
│  │       (discount from DB, not UI)    │       invoice_items      │
│  └─────────────────────────────────────┘                          │
│                  │                                                │
│                  ▼                                                │
│  ┌─────────────────────────────────────┐                          │
│  │  RECEPTIONIST (Billing)             │                          │
│  │  1. Sees unpaid invoice             │                          │
│  │  2. Collects payment                │──→ DB: invoices          │
│  │     (full or partial)               │       (status: Paid/Partial)│
│  │  3. Can void if needed              │                          │
│  └─────────────────────────────────────┘                          │
│                  │                                                │
│                  ▼                                                │
│ PATIENT LEAVES  ✓                                                 │
│                                                                   │
│  ┌─────────────────────────────────────┐                          │
│  │  ADMIN (Oversight — anytime)        │                          │
│  │  • Dashboard: real-time KPIs        │                          │
│  │  • Analytics: revenue, performance  │                          │
│  │  • Activity Log: full audit trail   │                          │
│  │  • Settings: cleanup, discounts     │                          │
│  │  • Employee Management + Schedules  │                          │
│  └─────────────────────────────────────┘                          │
│                                                                   │
│  ┌─────────────────────────────────────┐                          │
│  │  HR (Personnel — independent)       │                          │
│  │  • Approve/Decline leave requests   │                          │
│  │  • Manage employee records + salary │                          │
│  │  • Payroll & staffing reports       │                          │
│  │  • Auto-cancel appointments on leave│                          │
│  └─────────────────────────────────────┘                          │
└───────────────────────────────────────────────────────────────────┘
```

KEY TAKEAWAYS FOR DEFENSE

1. **Every action has a database trail** — from patient registration to payment, every step is an SQL operation that can be traced in the activity log.
2. **The flow mirrors real walk-in clinics** — patients arrive, get registered, triaged by a nurse (vitals auto-set status to "Triaged"), seen by a doctor (who prefers triaged patients), and pay at the front desk. The Triaged status creates a real Nurse → Doctor handoff pipeline.
3. **Each role does exactly one job well** — Receptionist handles intake and billing, Nurse handles triage (Waiting → Triaged), Doctor handles consultation (Triaged → In Progress → Completed), HR handles personnel, Admin oversees everything.
4. **Automation reduces errors** — appointments auto-confirm, auto-sync to queue, auto-create invoices, auto-apply discounts from database, auto-cancel during leave, auto-expire leave status.
5. **Security is layered** — role access at sidebar level, button level, and SQL level. Discounts enforced from database. Doctor data isolation. Full audit trail.

Study this document, trace through the code yourself, and practice explaining each feature out loud. Good luck on your defense!